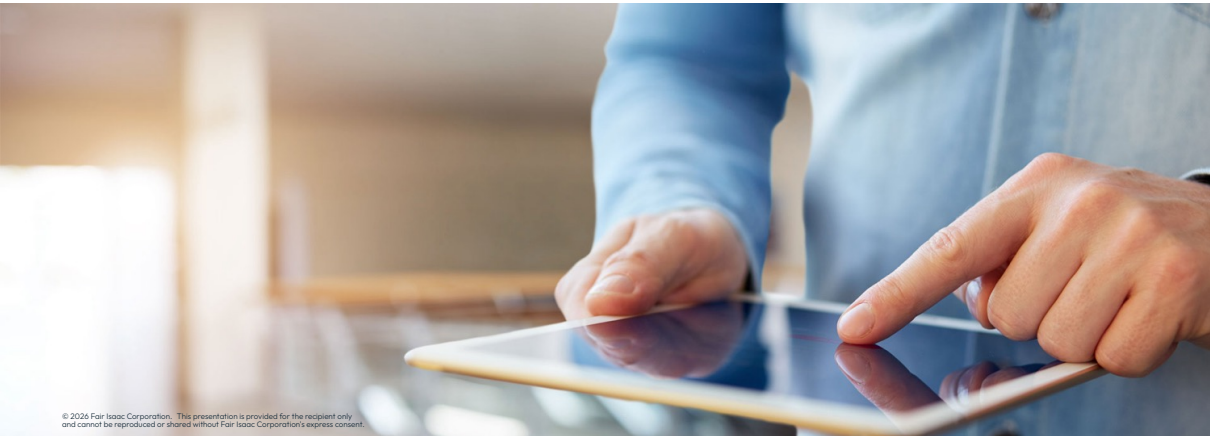
Using callbacks



Using callbacks

Using callbacks

- The library callbacks are a collection of functions which allow user–defined routines to be specified to the FICO® Xpress Optimizer:
  - Called at various stages during the optimization process, prompting the Optimizer to return to the user's program before continuing with the solution algorithm
  - Names of functions for defining callbacks are of the form problem.add*Callback()

Using callbacks

- The library callbacks are a collection of functions which allow user–defined routines to be specified to the FICO® Xpress Optimizer:
  - Called at various stages during the optimization process, prompting the Optimizer to return to the user's program before continuing with the solution algorithm
  - Names of functions for defining callbacks are of the form problem.add*Callback()
- Types of callbacks:
  - *Output callbacks*: called every time a text line is output by the Optimizer
    - The foremost use case, used for logging/reporting via the callback p.addMessageCallback()
  - *LP callbacks*: functions associated with the search for an LP solution
    - The functions p.addLplogCallback() and p.addBarlogCallback() allow the user to respond after each iteration of either the simplex or barrier algorithms, respectively
  - *MIP tree search callbacks*: called at various points of the MIP tree search process
    - For example, when a MIP solution is found at a node of the Branch-and-Bound, the Optimizer will call a routine set by p.addPreIntsolCallback() before saving the new solution

Finding help: *Check the Xpress Optimizer callbacks reference webpage to learn more about the most used callbacks*

Using callbacks

- Steps for using callbacks:
  1. Define a callback function (say `myfunction`) that is to be run at certain points in time (i.e. every time the BB reaches a specific point)

     ```
     def myfunction(prob, data, ...):
      # user-defined routine here...
     ```

  2. Call the corresponding problem.add*Callback() method with `myfunction` as its argument

     ```
     p.addPreIntsolCallback(myfunction, data)   # assume data defined elsewhere
     ```

  3. Run the p.optimize() command that launches the appropriate solver

Using callbacks

- Steps for using callbacks:
  1. Define a callback function (say `myfunction`) that is to be run at certain points in time (i.e. every time the BB reaches a specific point)

     ```
     def myfunction(prob, data, ...):
      # user-defined routine here...
     ```

  2. Call the corresponding problem.add*Callback() method with `myfunction` as its argument

     ```
     p.addPreIntsolCallback(myfunction, data)   # assume data defined elsewhere
     ```

  3. Run the p.optimize() command that launches the appropriate solver
- A callback function is passed once as an argument and used possibly many times while a solver is running, and receives:
  - A `problem` object declared with `p = xp.problem()`
  - A user-defined `data` object to read and/or modify information within the callback

Note: *The callbacks in the Python interface reflect as closely as possible the design of the callback functions in the C API*

Using callbacks

- Any call to a problem.add*Callback() function adds that function to a list of callback functions for that specific point of the BB algorithm:

```
p.addPreIntsolCallback(preint1, data, 3)
p.addPreIntsolCallback(preint2, data, 5)
```

  - The two functions will be put in a list and called (`preint2` first since it has a higher priority) whenever the BB algorithm finds an integer solution

Using callbacks

- Any call to a problem.add*Callback() function adds that function to a list of callback functions for that specific point of the BB algorithm:

```
p.addPreIntsolCallback(preint1, data, 3)
p.addPreIntsolCallback(preint2, data, 5)
```

  - The two functions will be put in a list and called (`preint2` first since it has a higher priority) whenever the BB algorithm finds an integer solution
- To remove a callback function, use the corresponding problem.remove*Callback() method:

```
p.remove*Callback(function,data)
```

  - Deletes all elements of the list of callbacks that were added with the corresponding add*Callback() function that match the function and the data, for example problem.removePreIntsolCallback()
  - The `None` keyword acts as a wildcard that matches any function or data object:
    - If `None` is passed as the callback function, then all callbacks matching the `data` argument will be deleted
    - If data is also `None`, all callback functions of that type are deleted, this can also be obtained by passing no argument to p.remove*Callback()

Using callbacks

- Example for a callback function named `preintsolcb` that is called every time a new integer solution is found via the p.addPreIntsolCallback() method:

```
import xpress as xp

def preintsolcb(prob, data, soltype, cutoff):
  # callback to be used when an integer solution is found defined here
  ...
  return (reject, newcutoff)  # assume 'reject' and 'newcutoff' defined meanwhile

p = xp.problem()
p.read('myprob.lp')  # reads in a problem, let's say a MIP

p.addPreIntsolCallback(preintsolcb, data)  # assume 'data' defined elsewhere
p.optimize()
```

Note: *While the* `function` *argument is necessary for all p.add*Callback() functions, the* `data` *object can be specified as* `None`*. In that case, the callback will be run with* `None` *as its data argument*

Thank You

www.fico.com/optimization